ChavScript — Especificação da Linguagem

Trabalho de Compiladores — "Do Meme ao Código"

1. Nome e meme escolhido

ChavScript — linguagem inspirada em **Chaves (El Chavo del Ocho)**, o personagem de Roberto Gómez Bolaños (Chespirito), ícone da cultura da internet brasileira e estrela da *Vila do Chaves*.

Justificativa

Chaves é um dos memes/personagens mais reconhecíveis do Brasil: frases como "O senhor tem alguma coisa contra a panela?", "É um canalha!", "Quero mais dinheiro!" e a famosa história do aluguel do Seu Madruga fazem parte do imaginário coletivo. A Vila do Chaves já ensinava português e matemática para crianças — a linguagem mantém esse espírito pedagógico: os comandos são as falas e ações dos personagens, e o "programa" é uma peça encenada no estúdio da Vila.

Assim como o BIRL transforma bordões do Bambam em comandos, o ChavScript transforma as interações da Vila em instruções de programação — sem copiar a sintaxe do BIRL.

2. Arquitetura do tradutor

O arquivo `ChavScript.html` é um tradutor completo que roda no navegador e implementa todas as fases de um compilador:

Fase	Função	O que faz

Análise léxica	`lex()`	Reconhece tokens (palavras reservadas, identificadores, strings, números, operadores, delimitadores), remove comentários `//` e aponta símbolos inválidos com linha.
Análise sintática	`parse()`	Descendente recursivo; produz uma **AST** a partir da gramática EBNF. Aponta comandos incompletos, blocos sem fechamento, delimitadores ausentes.
Análise semântica	`sem()`	Tabela de símbolos por conversa; verifica variáveis declaradas, tipos de expressões, operadores válidos por tipo, personagem participante da cena etc.
Interpretador	`events()`	Gerador assíncrono de eventos que executa a AST em tempo real (variáveis, laços e condições avaliados em runtime — necessário para a entrada de dados).
Geração de código	`toJS()`	Emite um programa **JavaScript (Node.js)** equivalente, executável em qualquer máquina com Node.

O console do estúdio mostra cada fase (`[LEXER]`, `[PARSER]`, `[SEMANTIC]`, `[COMPILER]`, `[ENGINE]`) com erros detalhados por linha.

3. Palavras reservadas e comandos

Comando ChavScript	Equivalente	Exemplo
`VILA COMEÇA` / `VILA TERMINA`	início/fim do programa	`VILA COMEÇA`
`CONVERSA ENTRE X E Y [E Z ...] ... CENA TERMINA`	unidade de execução (cena; 2 ou mais personagens)	`CONVERSA ENTRE CHAVES E QUICO E CHIQUINHA`
`ASSUNTO "..."`	tema do diálogo automático	`ASSUNTO "dinheiro"`
`ESTILO ...`	tom da cena	`ESTILO DRAMÁTICO`
`personagem ENTRA` / `SAI`	direção de cena	`CHAVES ENTRA`
`personagem DIZ/RESPONDE expr`	saída de texto	`QUICO DIZ "Oi!"`
`personagem PENSA expr`	pensamento (sem voz)	`CHAVES PENSA "Será?"`

`personagem GRITA expr`	fala em destaque	`QUICO GRITA "Chega!"`
`personagem PERGUNTA expr EM var`	**entrada de dados**	`CHAVES PERGUNTA "Idade?" EM idade`
`TEXTO nome = expr` / `NUMERO nome = expr`	declaração de variável	`NUMERO moedas = 2`
`SE expr OP expr ... SENÃO ... FIMSE`	condicional	`SE n MAIOR 1`
`REPETE expr VEZES ... FIM`	laço de repetição	`REPETE n VEZES`
`// comentário`	comentário de linha	`// isto é um comentário`

Personagens válidos: `CHAVES`, `SEU_MADRUGA`, `QUICO`, `CHIQUINHA`, `PROFESSOR_GIRAFALES`, `SR_BARRIGA`.

Cada `CONVERSA` aceita **dois ou mais personagens** (separados por `E`, sem repetição). Na conversação automática (sem falas escritas), o motor alterna as falas entre os **dois primeiros** personagens do elenco; com falas escritas, qualquer personagem do elenco pode falar.

Conversa automática

Se a conversa não contém nenhuma fala escrita (`DIZ`, `RESPONDE`, `PENSA`, `GRITA`), o motor gera o diálogo a partir do `ASSUNTO` e do `ESTILO` — um recurso cômico que usa as frases prontas da Vila.

4. Tipos de dados e expressões

- **`NUMERO`** — inteiros e decimais.
- **`TEXTO`** — strings entre aspas duplas.

Expressões com precedência usual:

```
expr    ::= termo (('+'|'-') termo)*
termo   ::= fator (('*'|'/') fator)*
fator   ::= '-' fator | primario
primario::= NUMERO | TEXTO | variavel | '(' expr ')'
```

Regras de tipo:

- `+` entre dois `NUMERO` → `NUMERO`; se houver `TEXTO`, concatena → `TEXTO`.
- `-`, `*`, `/` exigem `NUMERO`.
- Operadores relacionais no `SE`: `IGUAL`, `DIFERENTE` (qualquer tipo, desde

que iguais), `MAIOR`, `MENOR` (só `NUMERO`).

5. Gramática formal (EBNF)

```
programa    ::= VILA COMECA conversa* VILA TERMINA
conversa    ::= CONVERSA ENTRE personagem (E personagem)+
elenco      ::= 2 ou mais personagens, sem repetição
                ( ASSUNTO expr | ESTILO estilo | comando )*
                CENA TERMINA
comando     ::= personagem ENTRA | personagem SAI
              | personagem (DIZ|RESPONDE|PENSA|GRITA) expr
              | personagem PERGUNTA expr EM variavel
              | declaracao | repeticao | condicao
personagem  ::= CHAVES | SEU_MADRUGA | QUICO | CHIQUINHA | PROFESSOR_GIRAFALES | SR_BARRIGA
estilo      ::= ENGRACADO | EDUCATIVO | DRAMATICO
expr        ::= termo (('+'|'-') termo)*
termo       ::= fator (('*'|'/') fator)*
fator       ::= '-' fator | primario
primario    ::= NUMERO | TEXTO | variavel | '(' expr ')'
declaracao  ::= (TEXTO | NUMERO) nome = expr
repeticao   ::= REPETE expr VEZES comando* FIM
condicao    ::= SE expr (IGUAL|DIFERENTE|MAIOR|MENOR) expr comando*
                [ SENAO comando* ] FIMSE
```

Acentos são opcionais (`COMEÇA` = `COMECA`, `SENÃO` = `SENAO`).

`//` sempre inicia comentário — em divisões use espaços (`10 / 2`, não `10//2`).

6. Tokens

Token	Exemplos
Palavra reservada	`VILA`, `COMECA`, `CONVERSA`, `ENTRE`, `ASSUNTO`, `ESTILO`, `CENA`, `TERMINA`, `ENTRA`, `SAI`, `DIZ`, `RESPONDE`, `PENSA`, `GRITA`, `PERGUNTA`, `EM`, `TEXTO`, `NUMERO`, `SE`, `SENAO`, `FIMSE`, `IGUAL`, `DIFERENTE`, `MAIOR`, `MENOR`, `REPETE`, `VEZES`, `FIM`, `E`
Personagem	`CHAVES`, `SEU_MADRUGA`, `QUICO`, `CHIQUINHA`, `PROFESSOR_GIRAFALES`
Identificador	`[A-Za-zÀ-ÿ_][A-Za-zÀ-ÿ_0-9]*` (normalizado sem acento, maiúsculas)
String	`"texto entre aspas"`
Número	dígitos (`42`, `3.14` não suportado — use inteiros ou expressões)
Operador	`+ - * / = ( )`
Comentário	`//` até o fim da linha

7. Linguagem de destino: JavaScript (Node.js)

O botão **Gerar JavaScript** (ou `toJS()`) produz um programa Node.js:

- `TEXTO`/`NUMERO` → `let nome = expr;`
- `DIZ`/`RESPONDE` → `console.log("Personagem: " + texto)`
- `PENSA` → `console.log(" (Personagem pensa: " + texto + ")")`
- `GRITA` → `console.log("Personagem GRITA: " + texto)`
- `PERGUNTA` → `nome = __pergunta("...")` ou `__lerNumero("...")` (lê stdin)
- `SE` → `if/else` com `===`, `!==`, `>`, `<`
- `REPETE` → `for (let __i = 0; __i < n; __i++)`
- `ENTRA`/`SAI` → comentários de direção de cena
- Conversa automática → laço que imprime as falas do banco de frases

O código gerado lê as respostas do `PERGUNTA` pela entrada padrão:

```
node programa.js < respostas.txt
# ou cole as respostas no terminal e pressione Ctrl+D
```

8. Exemplo — recriação de cena real

O programa `testes/04_dinheiro_perdido.chav` (e o exemplo padrão do estúdio) recria a cena **"O Dinheiro Perdido"** (*El sobre de la renta*, Chaves, 1978; exibida no SBT e disponível no HBO Max/YouTube): o Seu Madruga perde o envelope com o dinheiro do aluguel, o Chaves encontra cem reais no bolso e tem de aguentar a sequência de socos — "foi sem querer querendo" / "agora eu vou te bater por querer querendo". O elenco reúne **todos os personagens do episódio** — Chaves, Chiquinha, Quico, Seu Madruga e Sr. Barriga — numa única `CONVERSA` com cinco participantes (`CONVERSA ENTRE CHAVES E CHIQUINHA E QUICO E SEU_MADRUGA E SR_BARRIGA`).

O programa usa `PERGUNTA` para o público informar onde o dinheiro foi encontrado, `SE ... IGUAL` para desviar o enredo, `REPETE` para a sequência de golpes e `DIZ/RESPONDE/PENSA/GRITA` para as falas da cena.

Trecho do JavaScript gerado:

```
// Conversa: Chaves e Seu Madruga
{
  let SOCOS = 4;
  let LUGAR = "";
  // Chaves entra em cena.
  // Seu Madruga entra em cena.
  console.log("Seu Madruga: "+("Eu juro que tinha o dinheiro dentro de um envelope!"));
  LUGAR = __pergunta(""+"Onde o senhor viu o dinheiro pela última vez?");
  if((LUGAR === "bolso")){
    console.log("Chaves: "+("Encontrei! Cem reais aqui no bolso!"));
    console.log("Seu Madruga GRITA: "+("O que te interessa?! Mas o dinheiro é meu, hein?"));
    for(let __i=0;__i<(SOCOS);__i++){
      console.log("Seu Madruga: "+("Direita! Esquerda! Outra direita!"));
      console.log("Chaves: "+("Ai! Ai! Ai!"));
    }
    // ...
```

```
  }
}
```

Execução (`echo bolso | node 04_dinheiro_perdido.js`):

```
Seu Madruga: Eu juro que tinha o dinheiro dentro de um envelope!
Onde o senhor viu o dinheiro pela última vez? Chaves: Encontrei! Cem reais aqui no bolso!
Seu Madruga GRITA: O que te interessa?! Mas o dinheiro é meu, hein?
Chaves: Foi sem querer querendo...
Seu Madruga GRITA: E agora eu vou te bater por querer querendo!
```

9. Tratamento de erros

Fase	Exemplos de erro
Léxico	símbolo inválido (`@`, `#`), aspas sem fechar
Sintático	`FIMSE`/`FIM`/`CENA TERMINA` faltando, `esperado =`, comando inesperado
Semântico	variável não declarada, variável redeclarada, personagem inexistente, tipo incompatível em conta ou comparação, `PERGUNTA` em variável inexistente
Runtime	`PERGUNTA` em `NUMERO` com resposta não numérica → aviso e nova tentativa; laço limitado a 1000 iterações por segurança

Todos os erros informam o número da linha.

10. Como usar

1. Abra `ChavScript.html` no navegador.

2. Escolha um exemplo ou escreva um programa.

3. **Compilar** mostra as fases; **Executar** encena a conversa (com entrada

interativa nas `PERGUNTA`); **Gerar JavaScript** produz o código Node na

aba **JS**, com botões de copiar e baixar.

4. Testes automatizados: `node testes/testar.js` (roda todos os `.chav` de

`testes/`, gera os `.js` em `testes/saida/` e executa cada um).